

CAR RENTAL MONOLITH

Vehicle Module Jira Backlog

Danh sách task đề xuất để tạo Jira issue cho module Vehicles, dựa trên source backend và admin-web hiện tại.

Workspace

C:/source/car-rental-monolith

Ngày tạo

22/05/2026

Phạm vi

Vehicle backend, admin UI, seed data, module kế tiếp

1. Tóm Tắt Hiện Trạng

Kết luận chính: module vehicle đã có API list, paged list và detail cơ bản, nhưng response còn mỏng so với dữ liệu seed và nhu cầu UI. Việc nên làm tiếp là hoàn thiện read model cho admin UI trước khi làm create/update.

Area	Hiện trạng	Khoảng trống chính
Backend vehicle	Có /admin/vehicles, /admin/vehicles/paged và /admin/vehicles/{id}. Detail mới enrich ownerCustomerName.	Chưa trả listing, images, features, availability, fleet asset, branch, price plan, recent bookings.
Database/seed	Seed đã tạo dữ liệu liên quan qua identity, customer, fleet, driver, vehicle, pricing, booking, payment, review, notification, admin.	Cần verification queries để phát hiện data cũ/orphan trong DB local.
Admin web vehicles	Vehicle page đã gọi backend thật, có grid/list, filter q/source/status, detail modal.	Type còn mỏng, text tiếng Việt bị lỗi encoding, card/detail chưa hiển thị data enrich.
Các module khác	Drivers page vẫn dùng mock. Sidebar có nhiều menu nhưng route thật mới có vehicles và drivers.	Nên chưa build booking ngay. Làm vehicle read model, sau đó customer/fleet read module.

2. Thứ Tự Làm Đề Xuất

Phase	Mục tiêu	Tasks
Phase 0	Ổn định UI và dữ liệu test	VEH-001, VEH-002
Phase 1	Hoàn thiện read model cho Vehicle Admin	VEH-003 đến VEH-009
Phase 2	Thêm admin actions cho status/listing	VEH-010 đến VEH-013
Phase 3	Create/update workflow	VEH-014 đến VEH-016
Phase 4	Test, build checks, API docs	VEH-017 đến VEH-019
Phase 5	Module liên quan sau Vehicle	VEH-020 đến VEH-022

3. Sprint Đầu Tiên Nên Chọn

Nếu muốn mở khóa UI nhanh nhất, sprint đầu nên lấy: VEH-001, VEH-002, VEH-003, VEH-004, VEH-006, VEH-007, VEH-008, VEH-009. Sau nhóm này, màn Vehicles sẽ nhìn giống một admin tool thật hơn thay vì chỉ hiển thị bảng xe cơ bản.

4. Jira Task List

VEH-001

Bug

P0

Frontend

Fix Vietnamese text encoding in admin web

Several admin-web labels render as mojibake, for example vehicle status labels and modal text. This makes the UI look broken before any new feature is added.

Scope	Replace corrupted display strings in vehicles, layout, sidebar, header and drivers feature with valid UTF-8 Vietnamese text.
Dependencies	None
Acceptance criteria	- Vehicle page labels render correctly in Vietnamese. - Sidebar, header and modal loading/error strings no longer show mojibake. - No API contract changes are required. - Vite build still passes.
Notes	Do this early because every subsequent UI task depends on readable labels.

VEH-002

Task

P0

Database

Add seed data verification queries for vehicle-related data

The seed script now creates data across multiple schemas. Add a lightweight SQL verification file so the team can quickly check orphan references and row counts after running seed.

Scope	Create documentation or SQL snippets for checking vehicle owner_customer_id, fleet_vehicle_id, bookings, payments and reviews.
Dependencies	Seed script already exists.
Acceptance criteria	- A query exists to detect HOST_OWNED vehicles whose owner_customer_id has no matching customer. - A query exists to detect COMPANY_OWNED vehicles whose fleet_vehicle_id has no matching fleet.company_vehicles row. - A row-count summary query exists for vehicle, listing, booking, payment and review tables. - The instructions say when to run the seed and verification.
Notes	This reduces confusion when old manual rows remain in the database.

VEH-003

Story

P0

Backend

Define admin vehicle list and detail API response contracts

Current VehicleResponse is too thin for admin UI. Split list and detail contracts so list stays compact while detail can include nested sections.

Scope	Add AdminVehicleListItemResponse and AdminVehicleDetailResponse, plus matching application view records.
Dependencies	Existing vehicle controller and VehicleDetail.
Acceptance criteria	- Paged list returns list-item fields only. - Detail endpoint returns nested sections for listing, owner, host, fleet, images, features, availability and recent bookings. - Existing ApiResponse and PageResponse wrappers remain unchanged. - Frontend can distinguish list item type from detail type.
Notes	This is the foundation for most remaining vehicle UI work.

VEH-004

Story

P0

Backend

Build enriched vehicle paged list query

The list page needs owner name, listing status, location, price, cover image and summary counts. Current paged query only reads vehicle.vehicles.

Scope	Replace or supplement VehicleJpaRepository.search with a projection query joining vehicle_listings, cover image, customer, fleet and simple aggregates.
Dependencies	VEH-003
Acceptance criteria	- List item includes ownerCustomerName for HOST_OWNED rows. - List item includes assetCode and branchName for COMPANY_OWNED rows when present. - List item includes listingStatus, city, district and baseDailyRate. - List item includes coverImageUrl, featureCount, activeAvailabilityBlockCount and bookingCount. - Pagination metadata remains correct.
Notes	Prefer a projection DTO over loading entities then doing N+1 queries.

VEH-005

Story

P1

Backend

Extend vehicle list filters and sorting

Admin UI will need filters beyond q/source/status to make seeded data useful for testing and daily operations.

Scope	Extend PageVehiclesQuery and persistence search to support listingStatus, city, district, fuelType, transmission, seats, minRate, maxRate and sort.
Dependencies	VEH-004
Acceptance criteria	- Invalid enum filter values return INVALID_PARAMETER instead of silently being ignored. - Filters can be combined without breaking pagination. - Search still supports brand, model, version and license plate. - Default sort remains createdAt descending unless a supported sort is passed.
Notes	This also fixes the current loose enum parsing behavior.

VEH-006

Story

P0

Backend

Build rich vehicle detail read model

Detail endpoint currently enriches only ownerCustomerName. The UI detail modal should expose the full vehicle context from related tables.

Scope	Load and return vehicle basic info, owner summary, host profile, fleet/company vehicle summary, branch, listing, images, features, availability blocks, price plan and recent bookings.
Dependencies	VEH-003
Acceptance criteria	- GET /api/v1/admin/vehicles/{id} returns all detail sections. - Missing optional relations are represented as null or empty arrays, not as errors. - HOST_OWNED and COMPANY_OWNED rows both render meaningful detail data. - Not found still returns DomainException.notFound through RestExceptionHandler.
Notes	Keep this as an admin read model, not a domain aggregate.

VEH-007

Story

P0

Frontend

Update frontend vehicle types and API client for enriched responses

Frontend types currently assume one thin Vehicle type for both list and detail. Align the UI with the new backend contracts.

Scope	Create VehicleListItem, VehicleDetail, nested section types and update vehicleAPI.ts.
Dependencies	VEH-003, VEH-004, VEH-006
Acceptance criteria	- getVehiclesPage returns PageResponse<VehicleListItem>. - getVehicleById returns VehicleDetail. - TypeScript catches missing fields instead of relying on optional thin fields everywhere. - Existing list and detail pages compile with the new types.
Notes	This should land together with or immediately after backend response changes.

VEH-008

Story

P1

Frontend

Upgrade vehicle cards and list view with enriched data

Vehicle cards currently show only icon, name, plate and basic specs. The seed data includes richer display fields that should appear in admin list.

Scope	Show cover image, owner or fleet label, listing status, city, base daily rate and summary counts on grid/list cards.
Dependencies	VEH-004, VEH-007
Acceptance criteria	- Cards use coverImageUrl when available and a stable fallback when missing. - HOST_OWNED cards show owner/host information. - COMPANY_OWNED cards show fleet asset/branch information. - List view remains readable at desktop and mobile widths.
Notes	Avoid making card height unstable when optional fields are missing.

VEH-009

Story P1 Frontend

Upgrade vehicle detail modal into sectioned admin view

Detail modal currently shows simple field groups only. It should expose the related vehicle data seeded for testing.

Scope	Add sections or tabs for Overview, Listing, Media, Features, Availability, Fleet/Owner and Bookings.
Dependencies	VEH-006, VEH-007
Acceptance criteria	- Images render as thumbnails with cover image highlighted. - Features render as badges or compact list. - Availability blocks show reason, time range and booking id when present. - Recent bookings show code, customer snapshot, status and total amount. - Empty states are explicit for missing optional sections.
Notes	A tabbed modal is acceptable if the page becomes too long.

VEH-010

Story P1 Backend

Add admin vehicle status transition API

Admins need to approve, deactivate or suspend vehicles. Current backend is read-only.

Scope	Add PATCH /api/v1/admin/vehicles/{id}/status with explicit allowed transitions and validation.
Dependencies	VEH-006
Acceptance criteria	- Allowed transitions are defined in application layer. - Invalid transitions return VALIDATION_ERROR or CONFLICT. - updated_at changes when status changes. - Response returns the updated vehicle detail or list item. - Tests cover allowed and rejected transitions.
Notes	Keep command logic separate from read query service.

VEH-011

Story P1 Backend

Add admin listing status transition API

Vehicle lifecycle and listing lifecycle are separate. Admin should be able to publish, pause or reject listings independently from vehicle status.

Scope	Add PATCH /api/v1/admin/vehicles/{id}/listing-status for vehicle.vehicle_listings.status.
Dependencies	VEH-006
Acceptance criteria	- PUBLISHED, PAUSED, REJECTED and DRAFT states are supported according to defined rules. - published_at is set when listing becomes PUBLISHED. - Missing listing returns a clear domain error. - Response includes updated listing section.
Notes	This works with seeded listing statuses.

VEH-012

Story

P1

Frontend

Add frontend admin actions for status and listing status

After backend command APIs exist, expose safe admin actions in list/detail UI.

Scope	Add action buttons, confirmation dialogs, loading states and react-query invalidation.
Dependencies	VEH-010, VEH-011
Acceptance criteria	• Actions are visible only when valid for the current state. • Confirm dialog clearly states the target status. • On success, list and detail queries refresh. • On error, user sees an actionable message.
Notes	Use existing button and modal styles.

VEH-013

Task

P2

Backend

Add audit/outbox record for admin vehicle actions

Admin status changes should be traceable. The schema already has admin.audit_logs and common.outbox_events.

Scope	Record admin vehicle status/listing changes into admin.audit_logs and optionally common.outbox_events.
Dependencies	VEH-010, VEH-011
Acceptance criteria	• Audit log records action, target_type VEHICLE, target_id and before/after data. • Outbox event is created for important transitions if needed. • Failure to write audit is handled consistently with transaction behavior.
Notes	Authentication is not implemented yet, so use a temporary system/admin actor until identity is wired.

VEH-014

Story

P2

Backend

Add create host-owned vehicle workflow

Create should come after read/detail/action APIs. Host-owned vehicles require owner_customer_id and listing data.

Scope	Add command API to create vehicle.vehicles plus vehicle_listings, initial images and features for HOST_OWNED vehicles.
Dependencies	VEH-003, VEH-006, customer seed/data availability
Acceptance criteria	• Request validates owner customer exists. • License plate uniqueness conflict returns CONFLICT. • Vehicle and listing are created in one transaction. • Response returns created detail view.
Notes	Do not support all edge cases initially; keep MVP focused.

VEH-015

Story

P2

Backend

Add create company-owned vehicle workflow

Company-owned vehicle creation touches vehicle and fleet tables.

Scope	Add command API to create vehicle.vehicles plus fleet.company_vehicles and listing.
Dependencies	VEH-014 or shared command foundation, fleet branch data
Acceptance criteria	- Request validates branch exists. - fleet_vehicle_id points to created fleet.company_vehicles.id. - Asset code and license plate conflicts return CONFLICT. - Created detail includes fleet and branch section.
Notes	This should reuse validation patterns from host-owned create.

VEH-016

Story

P2

Backend

Add update listing, images and features APIs

Admin will need to edit display data without changing the core vehicle identity every time.

Scope	Add APIs to update listing fields, replace/reorder images and replace feature set.
Dependencies	VEH-006
Acceptance criteria	- Listing update validates price, location and status. - Image API supports cover image and sort order. - Feature replacement is idempotent. - Detail endpoint reflects updates immediately.
Notes	Keep image URL storage simple for now; no upload service yet.

VEH-017

Task

P1

Backend QA

Add backend tests for vehicle read and action APIs

Current backend has only contextLoads. Vehicle module needs tests before it grows further.

Scope	Add focused tests for query normalization, repository projections, controller success/error paths and status transitions.
Dependencies	VEH-004, VEH-006, VEH-010
Acceptance criteria	- Tests cover paged list filters and pagination metadata. - Tests cover detail not found. - Tests cover invalid enum/filter behavior. - Tests cover allowed and rejected status transitions.
Notes	Use Spring test slices or integration tests depending on available DB/test setup.

VEH-018

Task

P2

Frontend QA

Add frontend build checks and basic vehicle UI smoke coverage

The admin UI has multiple states and API-driven views. Add a minimum safety net around vehicle page rendering.

Scope	Run build in CI/local checklist and add lightweight component or route smoke tests if test tooling is introduced.
Dependencies	VEH-007, VEH-008, VEH-009
Acceptance criteria	- npm run build passes. - Vehicle list empty/loading/error states are manually or automatically verified. - Detail modal handles missing optional sections without crashing.
Notes	If no test framework is introduced yet, document a manual smoke checklist.

VEH-019

Task

P2

Documentation

Document vehicle admin API contract and sample payloads

As response models become richer, API examples are needed for frontend and future module work.

Scope	Add docs/api/vehicle-admin.md or OpenAPI notes for list, detail, filters and actions.
Dependencies	VEH-003, VEH-004, VEH-006, VEH-010
Acceptance criteria	- Docs include endpoint, query params, response shape and sample JSON. - Docs include status transition rules. - Docs include common error responses.
Notes	This can be converted to OpenAPI later.

VEH-020

Story

P2

Backend

Build customer read module for admin linking

Vehicle UI needs owner/customer details. Current vehicle module does a direct native lookup to customer.customers because customer module is not implemented.

Scope	Create customer read APIs for admin list/detail and replace direct CustomerNameQuery later with a cleaner internal port or API.
Dependencies	VEH-006
Acceptance criteria	- Admin can page/search customers. - Customer detail exposes roles, host profile, KYC and addresses. - Vehicle detail can link to customer detail page when frontend route exists.
Notes	This is the next module after vehicle read model stabilizes.

VEH-021

Story

P2

Backend

Build fleet read module for company-owned vehicles

Company-owned vehicles need branch, asset, maintenance, insurance and inspection views.

Scope	Create fleet admin read APIs and use them to support richer vehicle/fleet UI.
Dependencies	VEH-006
Acceptance criteria	- Admin can page company vehicles by branch/status. - Fleet detail shows maintenance, insurance and inspection records. - Vehicle detail can link to fleet asset detail.
Notes	Do after vehicle detail exposes the minimal fleet summary.

VEH-022

Story

P3

Full stack

Replace driver mock page with real driver API later

Driver page currently uses mock data. Seed now contains driver rows, but this is lower priority than vehicle because vehicle page is already connected to backend.

Scope	Add driver backend read APIs and connect admin-web driver page to real data.
Dependencies	Vehicle read model stabilized, driver schema seed available.
Acceptance criteria	- Driver page loads from backend instead of mockDrivers. - Driver filters/status summary reflect seeded data. - Mock data file is removed or kept only for tests/storybook.
Notes	Schedule after vehicle/customer/fleet read foundations.